MODULE 26 · DOMAIN 4 - SECURITY AND COMPLIANCE

Encryption and Secrets

KMS keys, key policies, grants and rotation, plus ACM certificates and Secrets Manager.

DOMAIN 4 of 5 EXAM WEIGHT 16% MODULE 26 of 35

admin@sthithapragna.com

Encryption on AWS is two questions: **who holds the key**, and **who may use it**. The second is answered by a resource policy that behaves unlike any other.

Three kinds of key

KEY TYPE	WHAT YOU CONTROL
AWS owned	Nothing. Shared across accounts, invisible to you, free
AWS managed (aws/service)	Nothing much. Visible in your account, rotates yearly, free to store
Customer managed	Everything: key policy, grants, rotation, deletion. Charged monthly

ON THE EXAM

Only a customer managed key gives you a policy you can write, rotation you can set, and the ability to deny access. That is why compliance answers always name it.

Envelope encryption

- KMS does not encrypt your data. It encrypts a **data key**, which encrypts your data.
- **GenerateDataKey** returns a plaintext key and an encrypted copy. You use the plaintext, store the encrypted one, and discard the plaintext.
- To read the data later you call **Decrypt** on the stored encrypted key, then decrypt locally with the result.
- KMS has a **4 KB limit** on direct encryption. Anything larger must use envelope encryption, which is why every AWS service does.

> ON THE EXAM
>
> This is why reading an encrypted object requires kms:Decrypt. The object's data key has to be unwrapped before anything can be read.

The key policy, and why it is different

- **Every KMS key has a key policy, and it is mandatory.** There is no key without one.
- **An identity policy alone cannot grant access to a key.** The key policy must first delegate to IAM.
- The default policy includes a statement giving the **account root full access**, and that statement is what enables IAM policies to work.
- Remove it and the key can only be used by principals the key policy names directly, which is a legitimate and very strict design.

> ON THE EXAM
>
> This is the single most common KMS surprise. Granting kms:Decrypt in an identity policy achieves nothing unless the key policy allows IAM to grant it.

Key policy or grant?

KEY POLICY

Durable and declarative

- Written once, reviewed, version controlled
- Names principals and conditions
- The primary access control on a key
- Changing it is a deliberate act

GRANT

Temporary and programmatic

- Created by an API call, often by an AWS service
- Granular: specific operations, specific conditions
- Can be retired or revoked when finished
- How services get short-lived access on your behalf

THE TIEBREAKER

Grants exist because AWS services need scoped, temporary key access without rewriting your key policy every time you create a resource.

Key rotation, updated

- **Automatic rotation** replaces the key material on a schedule. Since **April 2024** the period is configurable from **90 to 2,560 days**, default 365.
- **On-demand rotation** also arrived in 2024. It rotates immediately, does not disturb the automatic schedule, and is capped at **10 times per key**.
- **Old key material is retained**, so data encrypted before a rotation still decrypts. The key ID and ARN never change.
- **Symmetric encryption keys only.** Not asymmetric, not HMAC, not custom key stores. **AWS managed keys rotate yearly and cannot be changed.**

> ON THE EXAM
>
> "Rotate every 90 days" was impossible before April 2024 and is now a setting. Older study material still says rotation is annual and fixed.

Disabling and deleting keys

- **Disabling** is immediate and reversible. Everything encrypted with the key becomes unreadable until you re-enable it.
- **Deletion is not immediate.** A mandatory waiting period of **7 to 30 days**, defaulting to 30, during which the key is in PendingDeletion.
- **Deleting a key destroys every ciphertext encrypted under it, permanently.** There is no recovery of any kind.
- Disable first and watch for errors. CloudWatch and CloudTrail will show anything that was still using it.

> ON THE EXAM
>
> The waiting period exists because key deletion is one of very few genuinely irreversible actions on AWS. Alarm on ScheduleKeyDeletion.

Multi-Region keys and custom key stores

- **Multi-Region keys** share key material and key ID across Regions, so ciphertext encrypted in one Region decrypts in another with no cross-Region call.
- They are **not global**. Each replica is a distinct key resource with **its own key policy, grants and tags**.
- **CloudHSM key store**: key material lives in your own CloudHSM cluster, under your control.
- **External key store**: key material lives entirely **outside AWS**, in a key manager you operate. For the strictest sovereignty requirements.

> ON THE EXAM
>
> Multi-Region keys are the answer for cross-Region replication of encrypted data. Ordinary keys are Regional, so a replica in another Region cannot decrypt.

Constraining how a key may be used

- **kms:ViaService** restricts key use to requests arriving through a named service, such as s3.eu-west-1.amazonaws.com.
- That stops a principal calling KMS directly to decrypt data they should only reach through S3.
- **Encryption context** is additional authenticated data. It must match exactly on decrypt, and it appears in CloudTrail.
- **kms:CallerAccount** and **aws:PrincipalOrgID** restrict which accounts may use the key at all.

> ON THE EXAM
>
> Encryption context in CloudTrail is what turns key usage logs from "someone decrypted something" into "this bucket, this object, this request".

S3 encryption options

OPTION	NOTES
SSE-S3	AES-256, keys managed entirely by S3. The default on every new object
SSE-KMS	A KMS key you choose. Auditable in CloudTrail, and you control the policy
DSSE-KMS	Dual-layer: two independent layers of KMS encryption. For the strictest requirements
SSE-C	You supply the key with every request. Disabled by default on new buckets since April 2026
Client-side	Encrypted before it reaches AWS. S3 never sees plaintext

ON THE EXAM

SSE-KMS is the usual answer whenever a question mentions auditing key usage, controlling who can decrypt, or a compliance requirement.

S3 Bucket Keys

- Without them, S3 calls KMS **once per object operation**. At millions of objects that is a very large KMS bill.
- A **bucket-level key** is generated from KMS and held briefly by S3, then used to create data keys for many objects.
- This reduces KMS request costs by **up to 99 percent**, with no application change at all.
- **Not supported with DSSE-KMS.** Applies to new objects; existing ones need a copy, which S3 Batch Operations can do.

> ON THE EXAM
>
> "Our KMS bill is enormous and it is all S3 requests" is a bucket key. It is a checkbox, and it is the highest-leverage encryption cost answer there is.

EBS and RDS encryption rules

- **EBS encryption is set at creation.** You cannot encrypt an existing volume in place.
- The route is **snapshot, copy the snapshot with encryption enabled, create a volume from the copy**.
- **Default encryption** is an account and Region setting, so every new volume is encrypted without anyone remembering.
- **RDS encryption is also set at creation.** Encrypting an existing instance means snapshot, copy with encryption, then restore.

> ON THE EXAM
>
> Neither can be encrypted in place. Both follow the same snapshot-copy-restore pattern, and both mean downtime or a migration.

AWS Certificate Manager

- **Public certificates are free** when used with integrated services such as ALB, CloudFront and API Gateway.
- **DNS validation renews automatically** as long as the record stays in place. **Email validation requires a human every time.**
- **Certificates are Regional.** A certificate for **CloudFront must be in us-east-1**, whatever Region your origin is in.
- An ACM certificate attached to a load balancer **cannot expire unnoticed**, which removes an entire class of outage.

> ON THE EXAM
>
> DNS validation over email validation, every time. Email validation is why certificates still expire in organisations that thought they had automated renewal.

New: exportable public certificates

- **Announced June 2025.** ACM public certificates can now be marked **exportable**, giving you the certificate and its **private key**.
- That means ACM can issue certificates for **EC2 instances, containers and on-premises hosts**, not only integrated AWS services.
- Exportable certificates are **charged**, unlike ordinary public certificates, and you must be explicit at request time.
- **You take over renewal and redeployment.** The automatic renewal that makes ACM valuable does not extend to an exported copy.

> ON THE EXAM
>
> Every course written before mid-2025 says ACM public certificates cannot be exported. That was true, and it is now a request-time option.

Private certificates

- **AWS Private CA** issues certificates for internal names that no public authority would sign.
- For internal load balancers, service-to-service TLS, and mutual TLS between microservices.
- It carries a **monthly charge per CA** plus a charge per certificate. It is not a free alternative to public certificates.
- Clients must **trust your CA**, which means distributing the root, and that distribution is the real work.

ON THE EXAM

Public CA for anything a browser will see. Private CA for internal names and machine-to-machine trust you control end to end.

Secrets Manager rotation

- Rotation runs a **Lambda function** on a schedule. AWS provides **managed rotation** for RDS, Aurora, Redshift and DocumentDB.
- **Single user** strategy changes the one password in place. **Alternating users** keeps two, rotating one while the other serves traffic.
- Alternating users gives **zero downtime rotation**, because there is always a valid credential in use.
- Rotation changes the credential **in the database and in the secret together**. That atomicity is the whole point.

> ON THE EXAM
>
> Module 7's tiebreaker restated: expiry deletes a value, rotation replaces it in two places at once. Only Secrets Manager does the second.

Secrets across accounts and Regions

- **Replication** copies a secret to other Regions, keeping replicas in sync when the source rotates.
- That is what makes a secret available in a disaster recovery Region, which module 17 listed as a common gap.
- **Cross-account access** needs a **resource policy on the secret** and permission on the **KMS key**, because a secret is always encrypted.
- A secret encrypted with the **default aws/secretsmanager key cannot be shared cross-account**. Use a customer managed key.

> ON THE EXAM
>
> The last point is a real design constraint. If a secret will ever be read from another account, it must use a customer managed key from the start.

> An application role has an identity policy allowing kms:Decrypt on a specific customer managed key, and s3:GetObject on the bucket. Downloads still fail with AccessDenied. Both the role and the key are in the same account.
>
> A security engineer previously removed the default statement from the key policy to tighten access.

What is the cause?

A. The key policy no longer delegates to IAM, so identity policies cannot grant access to the key

B. The role needs kms:Encrypt as well as kms:Decrypt

C. The key must be a multi-Region key

D. kms:Decrypt cannot be granted in an identity policy

◆ PAUSE AND DECIDE

What is the cause?

A Key policy no longer delegates to IAM

The default statement giving the account root full access is what allows identity policies to grant key access. Removing it makes the key policy the only authority.

B Needs kms:Encrypt too

Downloading decrypts. Encrypt is needed for writing, not reading, and adding it would not help.

C Needs a multi-Region key

Multi-Region keys matter when ciphertext crosses Regions. Everything here is in one account and one Region.

D Decrypt cannot be in an identity policy

It certainly can, provided the key policy delegates to IAM. That delegation is exactly what was removed.

THE SIGNAL

The setup tells you what changed. A KMS key policy is not like a bucket policy: without delegation, IAM grants are simply ignored for that key.

> A data platform stores 400 million small objects in S3 with SSE-KMS. The monthly bill shows a very large AWS KMS charge, dominated by request costs rather than key storage. Applications read objects continuously.
>
> The team must reduce cost without weakening encryption or changing application code.

What should the team do?

A Enable S3 Bucket Keys on the bucket

B Switch the bucket to SSE-S3

C Switch to DSSE-KMS with a bucket key

D Reduce the key's rotation frequency

◆ PAUSE AND DECIDE

What should the team do?

A Enable S3 Bucket Keys

Cuts KMS request traffic by up to 99 percent, keeps SSE-KMS and the customer managed key, and needs no application change.

B Switch to SSE-S3

Removes the KMS charge by removing KMS, so you lose the auditable key, the key policy and the control. That is weakening encryption.

C DSSE-KMS with a bucket key

Bucket keys are not supported with DSSE-KMS, and dual-layer doubles the work. It would increase cost, not reduce it.

D Rotate less often

Rotation is not billed per rotation, and the charge is per request. Rotation frequency has no bearing on it.

THE SIGNAL

"Dominated by request costs" is the phrase. The bill is per KMS API call, and a bucket key removes almost all of those calls.

> A compliance requirement states that the database encryption key must be rotated every 90 days. An engineer reports that AWS KMS only rotates keys annually and proposes writing a Lambda function to create a new key quarterly and re-encrypt all data.
>
> The database uses a customer managed symmetric key.

What should the team do?

A Build the Lambda function as proposed

B Enable automatic rotation with a 90-day rotation period

C Switch to an AWS managed key, which rotates more frequently

D Use a multi-Region key, which supports custom rotation

◆ PAUSE AND DECIDE

What should the team do?

A. Build the Lambda function

Solving a problem that no longer exists, with a function that re-encrypts a whole database quarterly. Considerable risk for nothing.

B. Automatic rotation, 90-day period

Since April 2024 the rotation period is configurable from 90 to 2,560 days on customer managed symmetric keys. This is a single setting.

C. AWS managed key

Rotates yearly and cannot be changed, so it is further from the requirement, not closer. It also removes your control of the policy.

D. Multi-Region key

Concerns where ciphertext can be decrypted, not how often material rotates. Irrelevant to the requirement.

THE SIGNAL

The engineer's premise was true until April 2024. The whole scenario turns on a fact that changed, which is why the currency matters.

> An application on EC2 in a private subnet needs a TLS certificate for an internal hostname. The team wants automatic renewal and does not want to manage a certificate authority. They have used ACM before, for load balancers.
>
> The certificate must be installed directly on the instances, which terminate TLS themselves.

What are the realistic options?

A. An ACM exportable public certificate, if the hostname is publicly resolvable, or AWS Private CA for an internal-only name

B. An ordinary ACM public certificate, exported to the instances

C. Terminate TLS at an ALB instead, using an ordinary ACM certificate

D. A self-signed certificate, since the subnet is private

◆ PAUSE AND DECIDE

What are the realistic options?

A Exportable public cert, or Private CA

Both put a certificate on the instance. Which applies depends on whether the name is publicly resolvable, since a public CA will not sign an internal-only name.

B Export an ordinary ACM certificate

The trap. Ordinary ACM public certificates cannot be exported; exportability must be chosen when the certificate is requested.

C Terminate at an ALB

A good design in general, and the scenario states the instances terminate TLS themselves. It answers a different question.

D Self-signed

No automatic renewal, and every client must be configured to trust it. That is running a CA badly rather than not running one.

THE SIGNAL

The exportable option must be selected at request time. An existing ACM certificate cannot be made exportable afterwards.

> An application in account A must read a database credential stored in Secrets Manager in account B. The secret is encrypted with the default aws/secretsmanager key. A resource policy has been added to the secret allowing the role in account A.
>
> Requests still fail with AccessDenied.

What is the cause?

A. A secret encrypted with the default AWS managed key cannot be shared across accounts. It must use a customer managed key

B. The secret must be replicated into account A

C. Cross-account access to Secrets Manager is not supported

D. The role in account A needs secretsmanager:CreateSecret

◆ PAUSE AND DECIDE

What is the cause?

A Default managed key blocks cross-account

A secret is always encrypted, so reading it requires decrypt on the key. AWS managed key policies cannot be edited to permit another account.

B Replicate the secret

Replication puts a copy in another Region, not another account. It does not address cross-account access at all.

C Not supported

It is fully supported, via a resource policy on the secret plus permission on a customer managed key.

D Needs CreateSecret

Reading a secret needs GetSecretValue. CreateSecret is for making new ones and would not help.

THE SIGNAL

Module 24's fourth rung again: the resource is only half of it. A secret is encrypted, so the key policy is always the second thing to check.

The five things to carry forward

- **An identity policy alone cannot grant key access.** The key policy must delegate to IAM first.
- **KMS rotation is configurable from 90 to 2,560 days** since 2024, plus on-demand. It is no longer annual and fixed.
- **S3 Bucket Keys cut KMS request costs by up to 99 percent**, with no application change.
- **ACM public certificates can be exportable** since June 2025, chosen at request time and charged.
- **Anything encrypted needs the key checked too.** Cross-account secrets require a customer managed key.

> ON THE EXAM
>
> Next: compliance and detection - Config, conformance packs, Security Hub, GuardDuty, Inspector and Macie.